

P0564R0: Wording for three-way comparisons

Introduction

This paper presents the detailed wording changes to implement P0515R0 "Consistent comparison" by Herb Sutter. Any differences in semantics to P0515R0 are unintentional.

Open Issues

- The rules in P0515R0 (options 2 or 3) when a compiler-declared operator<=> is deleted vs. not declared are not yet accurately reflected.

Wording

In 2.12 [lex.operators], add <=> as an option for the grammar non-terminal *preprocessing-op-or-punc*.

Add a new section 5.9 [expr.spaceship] before the existing 5.9 [expr.rel]:

5.9 Three-way comparison operator [expr.spaceship]

The three-way comparison operator groups left-to-right.

compare-expression:
 shift-expression
 compare-expression <=> *shift-expression*

If both operands have (possibly different) floating-point types, the usual arithmetic conversions are applied to the operands. The operator yields a prvalue of type `std::partial_ordering`. The expression `a <=> b` yields `std::partial_ordering::less` if `a` is less than `b`, `std::partial_ordering::greater` if `a` is greater than `b`, `std::partial_ordering::equivalent` if `a` is equivalent to `b`, and `std::partial_ordering::unordered` otherwise.

If both operands have the same enumeration type `E`: If `E` has more than one enumerator with a given value, the operator yields a prvalue of type `std::weak_ordering`, otherwise it yields a prvalue of type `std::strong_ordering`. In either case, the operator yields the result of converting the operands to the underlying type of `E` and applying <=> to the converted operands.

If at least one of the operands is a pointer, pointer conversions (4.11 [conv.ptr]), function pointer conversions (4.13 [conv.fctptr]), and qualification conversions (4.5 [conv.qual]) are performed on both operands to bring them to their composite pointer type (Clause 5 [expr]). If at least one of the operands is a pointer to member, pointer to member conversions (4.12) and qualification conversions (4.5) are performed on both operands to bring them to their composite pointer type (Clause 5). If both operands are null pointer constants, but not both of integer type, pointer conversions (4.11 [conv.ptr]) are performed on both operands to bring them to their composite pointer type (Clause 5 [expr]). In all cases, after the conversions, the operands shall have the same type. [Note: Array-to-pointer conversions (4.2 [conv.array]) are not applied. -- end note]

If the composite pointer type is a function pointer type, a pointer-to-member type, or `std::nullptr_t`, the operator yields a prvalue of type `std::strong_equality`; the operator yields `std::strong_equality::equal` if the (possibly converted) operands compare equal (5.10 [expr.eq]) and `std::strong_equality::unequal` if they compare unequal, otherwise the result of the operator is unspecified.

If the composite pointer type is an object pointer type, the operator yields the result of converting both operands to `std::uintptr_t` and comparing the converted operands using <=>, where the result is consistent with the result of equality and relational comparisons. [Note: That means, if two pointer operands `p` and `q` compare equal (5.10 [expr.eq]), `p <=> q` yields `std::strong_ordering::equal`; if `p` and `q` compare unequal, `p <=> q` yields `std::strong_ordering::less` if `q` compares greater than `p` and `std::strong_ordering::greater` if `p` compares greater than `q` (5.9 [expr.rel]). -- end note]

If both operands have the same integral type, the operator yields a prvalue of type `std::strong_ordering`. The

result is `std::strong_ordering::equal` if both operands are arithmetically equal, `std::strong_ordering::less` if the first operand is arithmetically less than the second operand, and `std::strong_ordering::greater` otherwise. [Note: Integral promotions (4.6 [conv.prom]) or integral conversions (4.8 [conv.integral]) are not applied. -- end note]

Otherwise, the program is ill-formed.

Change the grammar in 5.9(old) [expr.rel]:

```
relational-expression:  
  shift-expression compare-expression  
relational-expression < shift-expression compare-expression  
relational-expression > shift-expression compare-expression  
relational-expression <= shift-expression compare-expression  
relational-expression >= shift-expression compare-expression
```

Change in 5.20 [expr.const] paragraph 2:

- ...
- a three-way comparison (5.9(new) [expr.spaceship]) comparing pointers that do not point to subobjects of the same complete object;
- a relational (5.9) or equality (5.10) operator where the result is unspecified; or
- ...

Add a new section to clause 12 [special]:

12.9 Comparisons [class.compare]

A defaulted comparison operator function (5.9(new) [expr.spaceship], 5.9 [expr.rel], 5.10 [expr.eq]) for some class C shall be a non-template function declared in the *member-specification* of C that

- is a non-static member of C having one parameter of type `const C&` or
- a static member or friend of C having two parameters of type `const C&`,

in all cases naming the injected-class-name.

12.9.1 Three-way comparison [class.spaceship]

For a defaulted three-way comparison operator function, the declared return type shall be either `auto`, in which case the return type is deduced as described below, or one of the category types, in which case a value of the deduced return type shall be implicitly convertible to the declared return type.

The direct base class subobjects of C, in the order of their declaration in the *base-specifier-list* of C, followed by the non-static data members of C, in the order of their declaration in the *member-specification* of C, form a list of subobjects. In that list, any subobject of array type is recursively expanded to the sequence of its elements, in the order of increasing subscript. Let x_i denote the i -th element in the expanded list of subobjects for an object x , where x_i is an lvalue if it is has reference type, and a const xvalue otherwise. [Note: This yields the same result as a class member access (5.2.5 [class.mem]) on `const c`. -- end note] The type of the expression x_i $\Leftrightarrow$ x_i is denoted by R_i . If any R_i is not a category type, the return type is `void` and the operator function is defined as deleted.

Otherwise, the return type R is deduced as follows:

- If the list of subobjects is empty, R is `strong_ordering`.
- Otherwise, if
 - at least one R_i is `weak_equality` OR
 - at least one R_i is `strong_equality` and at least one R_j is `partial_ordering` OR `weak_ordering`,R is `weak_equality`.
- Otherwise, if at least one R_i is `strong_equality`, R is `strong_equality`.
- Otherwise, if at least one R_i is `partial_ordering`, R is `partial_ordering`.
- Otherwise, if at least one R_i is `weak_ordering`, R is `weak_ordering`.
- Otherwise, R is `strong_ordering`.

The return value V of the three-way comparison operator function invoked with arguments x and y of the same type is determined by comparing corresponding elements x_i and y_i in the expanded lists of subobjects for x and

y and converting each of the resulting values to type R. Let i denote the first index where $x_i \neq y_i$ yields a result value different from $R_i::\text{equivalent}$; V is that result value converted to R. If no such index exists, V is `std::strong_ordering::equal` converted to R.

Editing note: The following implements default generation of operator<=>, thereby also providing all relational and equality operators.

For a class type T, a non-member operator<=> is implicitly declared as

```
friend auto operator<=>(const X&, const X&) noexcept = default;
```

(where X names the injected-class-name) if

- there is no declaration of a comparison operator (5.9(new) [expr.spaceship], 5.9 [expr.rel], 5.10 [expr.eq]) (including friend declarations) in the *member-specification* of T or a public base class of T
- T has no user-provided or deleted copy constructor (12.8.1 [class.copy.ctor]),
- the first parameter of all copy constructors of T has type `const T&`,
- T has no user-provided copy assignment operator (12.8.2 [class.copy.assign]),
- the parameter of all copy assignment operators of T has type `T` or type `const T&`, and
- T has no user-provided destructor (12.4 [class.dtor]).

If the definition of operator<=> would satisfy the requirements of a `constexpr` function (7.1.5 [dcl.constexpr]), the implicitly-declared operator<=> is `constexpr`.

12.9.2 Other comparison operators [class.rel.eq]

A defaulted relational (5.9 [expr.rel]) or equality (5.10 [expr.eq]) operator function for some operator @ shall have a declared return type `bool`.

The operator function with parameters x and y is defined as deleted if

- overload resolution (13.3), as applied to $x \neq y$ (also considering synthesized candidates with reversed order of parameters), results in an ambiguity or a function that is deleted or inaccessible from the operator function, or
- the operator @ cannot be applied to the return type of $x \neq y$ or $y \neq x$.

Otherwise, the operator function yields $x \neq y @ 0$ if an operator<=> with the original order of parameters was selected, or $0 @ y \neq x$ otherwise.

[Example:

```
struct C {
    friend std::strong_equality operator<=>(const C&, const C&);
    bool operator==(const C& x, const C& y) = default; // ok, returns x == y == 0
    bool operator<(const C&, const C&) = default;      // ok, function is deleted
};
```

-- end example]

Change in 13.3.1.2 [over.match.oper] paragraph 6 and add a new paragraph after that:

The set of candidate functions for overload resolution is the union of the member candidates, the non-member candidates, and the built-in candidates, all for operator@. If the operator is a relational (5.9 [expr.rel]) or equality (5.10 [expr.eq]) operator, a member or non-member candidate operator<=> is added to the set of candidate functions for overload resolution if the candidate was user-declared and

- the candidate has return type `std::strong_ordering`, `std::weak_ordering`, OR `std::partial_ordering` OR
- the candidate has return type `std::strong_equality` OR `std::weak_equality` and @ is == or !=.

For each such added candidate whose parameter types differ, a synthesized candidate is added to the candidate set where the order of the two parameters is reversed.

The argument list contains all of the operands of the operator. The best function from the set of candidate functions is selected according to 13.3.2 and 13.3.3. [Footnote: ...] [Example: ... -- end example]

If overload resolution yields no viable function (13.3.2 [over.match.viable]) for a relational (5.9 [expr.rel]) or equality (5.10 [expr.eq]) operator, then overload resolution is attempted again, with implicitly-declared operator<=> functions added to the candidate set.

If a candidate for `operator<=>` is selected by overload resolution, but `@` is not `<=>`, the call to `operator@` with arguments `x` and `y` yields the value of `@ @ operator<=>(y,x)` if the selected candidate is a synthesized candidate with reversed order of parameters, or `operator<=>(x,y) @ 0` otherwise.

If a built-in candidate is selected by overload resolution, the operands of class type are converted to the types of the corresponding parameters of the selected operation function, except that ...

Add new bullets before bullet 6 in 13.3.3 [over.match.best] paragraph 1:

- ...
- **F1 is an operator function for a relational (5.9 [expr.rel]) or quality (5.10 [expr.eq]) operator and F2 is not [Example:**

```
struct S {
    auto operator<=>(const S&, const S&) = default; // #1
    bool operator<(const S&, const S&);           // #2
};
bool b = S() < S(); // calls #2
```

-- end example] or, if not that,

- **F1 and F2 are operator functions for `operator<=>` and F2 is a synthesized candidate with reversed order of parameters and F1 is not [Example:**

```
struct S {
    std::weak_ordering operator<=>(const S&, int); // #1
    std::weak_ordering operator<=>(int, const S&); // #2
};
bool b = 1 < S(); // calls #2
```

-- end example] or, if not that,

- F1 is generated from a deduction-guide (13.3.1.8) and F2 is not [Example: ...]

In 13.5 [over.oper] paragraph 1, add `<=>` as an option for the grammar non-terminal *operator*.

Add two new paragraphs after 13.6 [over.built] paragraphs 12:

For every integral type `T` there exist candidate operator functions of the form

```
std::strong_ordering operator<=>(T , T );
```

For every pair of floating-point types `L` and `R`, there exist candidate operator functions of the form

```
std::partial_ordering operator<=>(L , R );
```

Change in 13.6 [over.built] paragraphs 15 and 16:

For every `T`, where `T` is an enumeration type or a pointer type, there exist candidate operator functions of the form

```
bool operator<(T , T );
bool operator>(T , T );
bool operator<=(T , T );
bool operator>=(T , T );
bool operator==(T , T );
bool operator!=(T , T );
R operator<=>(T , T );
```

where `R` is the result type specified in 5.9 [expr.spaceship].

For every pointer to member type `T` or type `std::nullptr_t` there exist candidate operator functions of the form

```
bool operator==(T , T );
bool operator!=(T , T );
std::strong_equality operator<=>(T , T );
```

Add a new paragraph after 13.6 [over.built] paragraphs 16:

For every array type `T` there exist candidate operator functions of the form

```
R operator<=>(T& , T& );
R operator<=>(T&& , T&& );
```

where `R` is the result type specified in 5.9 [expr.spaceship].

Add a new paragraph after 15.4 [except.spec] paragraph 10:

A deallocation function (3.7.4.2) with no explicit `noexcept`-specifier has a non-throwing exception specification.

The exception specification for **an implicitly-declared three-way comparison operator, or a three-way comparison without a *noexcept-specifier*** that is defaulted on its first declaration, is potentially-throwing if and only if the invocation of any comparison operator in the implicit definition is potentially-throwing.